



گزارش کار آزمایش

نام درس:

آزمایشگاه طراحی سیستم های دیجیتال

عنوان آزمایش:

ALU اعداد مختلط

اعضای گروه:

کیمیا رضائی

کیمیا قدیر

ملینا طاهری

مقدمه:

در بسیاری از کاربردهای مهندسی و علمی، به‌ویژه در پردازش سیگنال و سیستم‌های مخابراتی، از اعداد مختلط استفاده می‌شود. یک عدد مختلط به صورت زیر نمایش داده می‌شود:

$$A = A_r + jA_i$$

$$B = B_r + jB_i$$

که در آن A_r و B_r بخش حقیقی و A_i و B_i بخش موهومی اعداد مختلط هستند.

هدف این آزمایش طراحی یک سیستم محاسباتی است که بتواند عملیات مختلف را روی اعداد مختلط انجام دهد. در این طراحی هر عدد مختلط به صورت یک داده ۱۶ بیتی نمایش داده می‌شود که شامل دو بخش ۸ بیتی است:

- ۸ بیت پرارزش: بخش حقیقی

- ۸ بیت کم‌ارزش: بخش موهومی

در این پروژه تلاش شده است با استفاده از مفهوم اشتراک منابع (Resource Sharing) تعداد واحدهای سخت‌افزاری مورد استفاده کاهش یابد. به همین منظور تنها از یک ضرب‌کننده و یک جمع‌کننده/تفریق‌کننده برای انجام تمامی محاسبات استفاده شده است.

فرمول عملیات روی اعداد مختلط

برای انجام عملیات ضرب بین دو عدد مختلط از رابطه زیر استفاده می‌شود:

$$(a + bi)(c + di) = (ac - bd) + (ad + bc)i$$

مطابق این رابطه برای محاسبه ضرب اعداد مختلط به چهار عملیات ضرب و دو عملیات جمع یا تفریق نیاز است. در این پروژه این عملیات‌ها به صورت مرحله‌ای و با استفاده از یک ماشین حالت متناهی انجام می‌شوند تا از یک ضرب‌کننده و یک جمع‌کننده مشترک استفاده شود.

ابزارها و محیط آزمایش

- زبان توصیف سخت‌افزار Verilog
- نرم‌افزار ModelSim برای شبیه‌سازی
- فایل متنی mem_file.txt برای مقداردهی اولیه حافظه دستور
- ماژول‌های مستقل برای طراحی ساختار ماژولار

معرفی ماژول های طراحی شده

۱) ماژول Adder_Subtractor

ماژول Adder_Subtractor برای انجام عملیات جمع و تفریق بین دو عدد طراحی شده است. این ماژول دارای دو ورودی A و B و یک سیگنال کنترلی به نام ctrl است.

اگر مقدار سیگنال ctrl برابر صفر باشد، عملیات جمع انجام می شود و اگر مقدار آن برابر یک باشد، عملیات تفریق صورت می گیرد. خروجی این ماژول نتیجه عملیات انجام شده روی دو ورودی است.

از این ماژول برای محاسبه بخش حقیقی و موهومی اعداد مختلط استفاده می شود. استفاده از یک واحد جمع/تفریق مشترک باعث کاهش تعداد واحدهای سخت افزاری مورد استفاده در سیستم شده است.

۲) ماژول Multiplier

ماژول Multiplier برای انجام عملیات ضرب بین دو عدد ۸ بیتی علامت دار طراحی شده است. ورودی های این ماژول شامل دو عدد ۸ بیتی A و B هستند و خروجی آن یک مقدار ۱۶ بیتی است که حاصل ضرب این دو عدد را نشان می دهد.

این ماژول به صورت ترکیبی طراحی شده است و هر زمان که ورودی ها تغییر کنند، خروجی نیز به صورت مستقیم به روزرسانی می شود. این ماژول در محاسبات ضرب اعداد مختلط در واحد ALU استفاده می شود.

۳) ماژول Instruction_Memory

برای ذخیره دستورات از یک حافظه دستور استفاده شده است. این حافظه دارای ۳۲ خانه است که هر خانه شامل یک دستور می باشد. آدرس دهی این حافظه با استفاده از ۵ بیت انجام می شود که امکان دسترسی به ۳۲ مکان مختلف حافظه را فراهم می کند.

دستورات اولیه در فایلی به نام mem_file.txt ذخیره شده اند و در زمان شبیه سازی با استفاده از دستور readmemb در حافظه بارگذاری می شوند.

در هر سیکل کلاک، دستور مربوطه از حافظه خوانده شده و برای اجرا به واحد محاسباتی ارسال می شود.

۴) ماژول Complex_ALU

ماژول Complex_ALU هسته اصلی سیستم محاسباتی است و وظیفه انجام عملیات روی اعداد مختلط را بر عهده دارد. این ماژول بر اساس کد عملیات (opcode) نوع عملیات مورد نظر را تعیین می کند.

برای پیاده سازی ضرب اعداد مختلط با استفاده از یک ضرب کننده و یک جمع کننده، از یک ماشین حالت متناهی (FSM) استفاده شده است. در هر حالت از FSM بخشی از عملیات انجام می شود. به عنوان مثال:

- در حالت های ابتدایی ضرب های لازم محاسبه می شوند.

- در حالت‌های بعدی نتایج ضرب‌ها با استفاده از جمع یا تفریق ترکیب می‌شوند.

- در نهایت بخش حقیقی و موهومی خروجی تولید می‌شود.

در پایان عملیات سیگنالی به نام **done** فعال می‌شود تا نشان دهد نتیجه آماده است.

۵) ماژول **Pipelined_Computer**

برای مدیریت اجرای دستورات در سطح سیستم از ماژولی به نام **Pipelined_Computer** استفاده شده است. این ماژول شامل اجزای اصلی زیر است:

- شمارنده برنامه (Program Counter)

- حافظه دستور

- واحد محاسباتی **Complex_ALU**

- واحد کنترل

در هر سیکل کلاک، یک دستور از حافظه خوانده می‌شود و برای اجرا به **ALU** ارسال می‌گردد. از آنجا که برخی عملیات‌ها مانند ضرب مختلط چند سیکل طول می‌کشند، سیگنالی به نام **done** از **ALU** ارسال می‌شود تا پایان عملیات را مشخص کند.

در صورتی که عملیات هنوز کامل نشده باشد، واحد کنترل با ایجاد حالت **Stall** از افزایش شمارنده برنامه جلوگیری می‌کند تا دستور بعدی قبل از پایان عملیات جاری اجرا نشود. این مکانیزم باعث جلوگیری از بروز خطاهای داده‌ای در سیستم می‌شود.

روند طراحی و پیاده سازی

در ابتدا ماژول‌های پایه شامل ضرب‌کننده و جمع‌کننده/تفریق‌کننده طراحی و آزمایش شدند. پس از اطمینان از صحت عملکرد این ماژول‌ها، واحد محاسباتی **Complex_ALU** طراحی شد که با استفاده از **FSM** عملیات پیچیده‌تر روی اعداد مختلط را انجام می‌دهد.

در مرحله بعد حافظه دستور طراحی و دستورات مورد نظر در فایل حافظه قرار داده شد. در نهایت با اتصال تمامی این بخش‌ها، سیستم کامل **Pipelined_Computer** پیاده‌سازی گردید.

شبیه سازی و نتایج

برای بررسی صحت عملکرد سیستم، برای هر ماژول یک تست‌بنچ جداگانه نوشته شد. در این تست‌بنچ‌ها مقادیر مختلفی به ورودی‌ها اعمال شد و خروجی‌ها بررسی شدند.

نتایج شبیه‌سازی نشان داد که:

- ماژول ضرب‌کننده به درستی عمل می‌کند.
 - ماژول جمع و تفریق نتایج صحیح تولید می‌کند.
 - ALU قادر است عملیات روی اعداد مختلط را به صورت مرحله‌ای انجام دهد.
 - مکانیزم Stall از اجرای همزمان دستورات جلوگیری می‌کند و باعث هماهنگی بین اجزای سیستم می‌شود.
- در نهایت سیستم قادر است دستورات موجود در حافظه را به ترتیب خوانده و عملیات مورد نظر را روی اعداد مختلط انجام دهد.

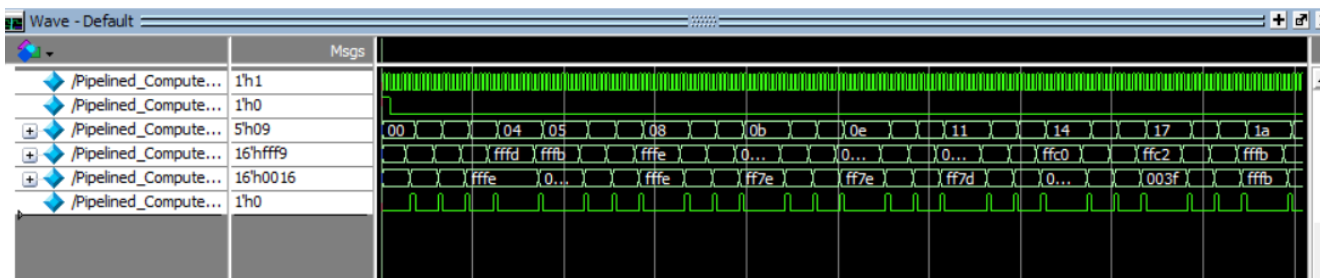
جمع بندی

در این آزمایش یک سیستم محاسباتی برای پردازش اعداد مختلط طراحی و پیاده‌سازی شد. در این طراحی با استفاده از مفهوم اشتراک منابع تنها یک ضرب‌کننده و یک جمع‌کننده/تفریق‌کننده برای انجام تمامی محاسبات مورد استفاده قرار گرفت.

استفاده از ماشین حالت متناهی برای کنترل مراحل مختلف محاسبه باعث شد عملیات پیچیده ضرب اعداد مختلط با حداقل منابع سخت‌افزاری انجام شود. همچنین با استفاده از حافظه دستور و مکانیزم Stall، اجرای صحیح دستورات در سطح سیستم تضمین گردید.

نتایج شبیه‌سازی نشان می‌دهد که سیستم طراحی شده قادر است عملیات مورد نظر را به درستی انجام دهد و اهداف آزمایش با موفقیت تحقق یافته‌اند.

تصاویر اجرای ماژول پایپ لاین هم در زیر آمده:



# 1755000	23	10		2	0		64	-63		-62	63		0
# 1765000	23	10		2	0		64	-63		128	63		0
# 1775000	23	10		2	0		64	-63		128	-126		1
# 1785000	24	10		2	0		64	-63		128	-126		0
# 1795000	24	00		127	-4		4	1		128	-126		0
# 1805000	24	00		127	-4		4	1		128	-126		0
# 1815000	24	00		127	-4		4	1		128	-126		0
# 1825000	24	00		127	-4		4	1		131	-126		0
# 1835000	24	00		127	-4		4	1		131	-3		1
# 1845000	25	00		127	-4		4	1		131	-3		0
# 1855000	25	01		-1	-4		4	1		131	-3		0
# 1865000	25	01		-1	-4		4	1		131	-3		0
# 1875000	25	01		-1	-4		4	1		131	-3		0
# 1885000	25	01		-1	-4		4	1		-5	-3		0
# 1895000	25	01		-1	-4		4	1		-5	-5		1
# 1905000	26	01		-1	-4		4	1		-5	-5		0
# 1915000	26	10		127	-4		4	1		-5	-5		0
# 1925000	26	10		127	-4		4	1		-5	-5		0
# 1935000	26	10		127	-4		4	1		-5	-5		0
# 1945000	26	10		127	-4		4	1		-5	-5		0
# 1955000	26	10		127	-4		4	1		-5	-5		0
# 1965000	26	10		127	-4		4	1		-5	-5		0
# 1975000	26	10		127	-4		4	1		-5	-5		0
# 1985000	26	10		127	-4		4	1		512	-5		0
# 1995000	26	10		127	-4		4	1		512	111		1
# 2005000	27	10		127	-4		4	1		512	111		0
# 2015000	27	00		0	0		0	0		512	111		0
# 2025000	27	00		0	0		0	0		512	111		0
# 2035000	27	00		0	0		0	0		512	111		0
# 2045000	27	00		0	0		0	0		0	111		0
# 2055000	27	00		0	0		0	0		0	0		1
# 2065000	28	00		0	0		0	0		0	0		0
# 2075000	28	00		1	2		3	4		0	0		0

# 2705000	5	10		3	5		6	7		-5	10	0
# 2715000	5	10		3	5		6	7		-5	10	0
# 2725000	5	10		3	5		6	7		-5	10	0
# 2735000	5	10		3	5		6	7		-5	10	0
# 2745000	5	10		3	5		6	7		-5	10	0
# 2755000	5	10		3	5		6	7		-5	10	0
# 2765000	5	10		3	5		6	7		-17	10	0
# 2775000	5	10		3	5		6	7		-17	51	1
# 2785000	6	10		3	5		6	7		-17	51	0
# 2795000	6	00		2	3		4	5		-17	51	0
# 2805000	6	00		2	3		4	5		-17	51	0
# 2815000	6	00		2	3		4	5		-17	51	0
# 2825000	6	00		2	3		4	5		6	51	0
# 2835000	6	00		2	3		4	5		6	8	1
# 2845000	7	00		2	3		4	5		6	8	0
# 2855000	7	01		2	3		4	5		6	8	0
# 2865000	7	01		2	3		4	5		6	8	0
# 2875000	7	01		2	3		4	5		6	8	0
# 2885000	7	01		2	3		4	5		-2	8	0
# 2895000	7	01		2	3		4	5		-2	-2	1
# 2905000	8	01		2	3		4	5		-2	-2	0
# 2915000	8	10		2	3		4	5		-2	-2	0
# 2925000	8	10		2	3		4	5		-2	-2	0
# 2935000	8	10		2	3		4	5		-2	-2	0
# 2945000	8	10		2	3		4	5		-2	-2	0
# 2955000	8	10		2	3		4	5		-2	-2	0
# 2965000	8	10		2	3		4	5		-2	-2	0
# 2975000	8	10		2	3		4	5		-2	-2	0
# 2985000	8	10		2	3		4	5		-7	-2	0
# 2995000	8	10		2	3		4	5		-7	22	1
# 3005000	9	10		2	3		4	5		-7	22	0
# 3015000	9	00		2	-127		2	3		-7	22	0

** Note: \$stop : D:/6/az/8/Pipelined Computer tb.v(33)